School of Computer Science
FACULTY OF ENGINEERING AND
PHYSICAL SCIENCES

UNIVERSITY OF LEEDS

COMP3011 Web Services and Web Data

Coursework 1 - Individual Web Services API Development Project

Technical Report: Green FinTech Banking-as-a-Service (BaaS) API

GitHub:
`https://github.com/scAB1001/green-fintech-baas`

Google Drive:
`https://drive.google.com/drive/folders/1Z7ckxHVmMphmtXQQRbI7SNM2GNpGRVgF?usp=sharing`

Andreas Bazigos — sc222ab
March 13, 2026

1 Introduction & Architectural Design

1.1 Project Overview & API Audience

The Green FinTech Banking-as-a-Service (BaaS) API is engineered to bridge the "green financing gap" by equipping commercial lenders with a computational backend to dynamically generate Sustainability-Linked Loan (SLL) quotes and meet Sustainability Performance Targets (SPTs) Practical Law Finance (2026). Operating as an open, public service Stoplight (2026), the API targets financial institutions, strictly adhering to the OpenAPI 3.1 standard OpenAPI Initiative (2023).

1.2 Architectural Research & System Justification

When architecting modern web services, full-stack paradigms such as MEAN (MongoDB, Express, Angular, Node.js) and MERN dominate agile development. However, evaluating these traditional architectures against the strict regulatory and computational requirements of a financial lending engine revealed issues.

Specifically, the NoSQL nature of MongoDB (found in MEAN/MERN) prioritises horizontal scalability and flexible schemas over relational integrity. For a BaaS application recording immutable loan ledgers and preventing duplicate ESG reporting, strict ACID (Atomicity, Consistency, Isolation, Durability) compliance is essential. While Node.js provides a capable asynchronous event loop, Python's ecosystem offers superior tooling for data manipulation (e.g., Pandas The pandas development team (2026)) and modelling.

Consequently, this project rejects the monolithic and NoSQL paradigms in favour of a composite REST architecture underpinned by Domain-Driven Design (DDD). The network routing layer is decoupled from the ESG mathematical engine and database persistence, yielding a highly testable microservice foundation.

1.3 Core Technology Stack Selection

The selected technology stack reflects a conscious prioritisation of asynchronous I/O, strict type safety, and deterministic deployment.

- **Language & Framework (Python 3.12 & FastAPI):** Traditional Python web frameworks like Django impose a heavyweight, monolithic structure Fırat Özcan (2024), while Flask requires third-party plugins to achieve basic asynchronous routing and OpenAPI support. FastAPI Sebastián Ramírez (2026) was selected for its native ASGI (Asynchronous Server Gateway Interface) capabilities via Uvicorn Encode OSS (2026). Paired with Python 3.12 Python Software Foundation (2024), it leverages Pydantic Samuel Colvin (2026) to provide strict payload validation at the network boundary, catching malformed requests before they reach the business logic.
- **Persistence Layer (PostgreSQL & SQLAlchemy 2.0):** To satisfy the requirement for a robust SQL database, PostgreSQL 18 The PostgreSQL Global Development Group (2026) was chosen for its enterprise-grade reliability. Interaction is managed by SQLAlchemy 2.0 Mike Bayer and SQLAlchemy authors (2026*b*) utilising the `asyncpg` driver MagicStack Inc. (2026) to unlock native asynchronous I/O Mike Bayer and SQLAlchemy authors (2026*a*). This async-first connection ensures that the FastAPI event loop is never blocked while awaiting complex disk I/O operations, an advantage over synchronous ORMs.
- **Package Management (uv):** Historically, Python environments suffer from non-deterministic dependency resolution using `pip` or slow locking via `Poetry` Hitorunajp (2025). This project utilises `uv` Astral Software Inc. (2026*b*), a cutting-edge, Rust-based package manager. It guarantees identical local and CI/CD environments in milliseconds, accelerating the deployment pipeline Loopwerk (2024).
- **Caching & Performance (Redis):** Generating dynamic SLL PDF quotes via ReportLab ReportLab Europe Ltd. (2026) is a CPU-bound operation that threatens to throttle asynchronous throughput. To mitigate this, a Redis Redis Ltd. (2026) caching tier was implemented. By utilising a Cache-Aside pattern Bindu B C (2024), Level Up Coding (2024) to store Base64 encoded binary streams and company directories, endpoint latency for read operations was reduced from $\sim$400ms to $<$ 10ms.

2 Database Design & Implementation

2.1 Relational Schema & Persistence

The platform employs PostgreSQL 18 The PostgreSQL Global Development Group (2026) accessed via SQLAlchemy 2.0 Mike Bayer and SQLAlchemy authors (2026*b*) with the `asyncpg` driver MagicStack Inc. (2026). This guarantees strict ACID compliance and referential integrity across all entities.

Following Domain-Driven Design (DDD) principles, the database schema is partitioned into a mutable Transactional Domain (TD) (`Company`, `EnvironmentalMetric`, `LoanSimulation`) and an immutable Macro Reference Dictionary (MRD) (`NationalEnergy`, `RegionalEmission`). Complete CRUD (Create, Read, Update, Delete) functionality is mapped strictly to the `Company` root aggregate. Hard-deletions trigger PostgreSQL's native `ON DELETE CASCADE`, instantly wiping orphaned metrics and SLL quotes to maintain data hygiene.

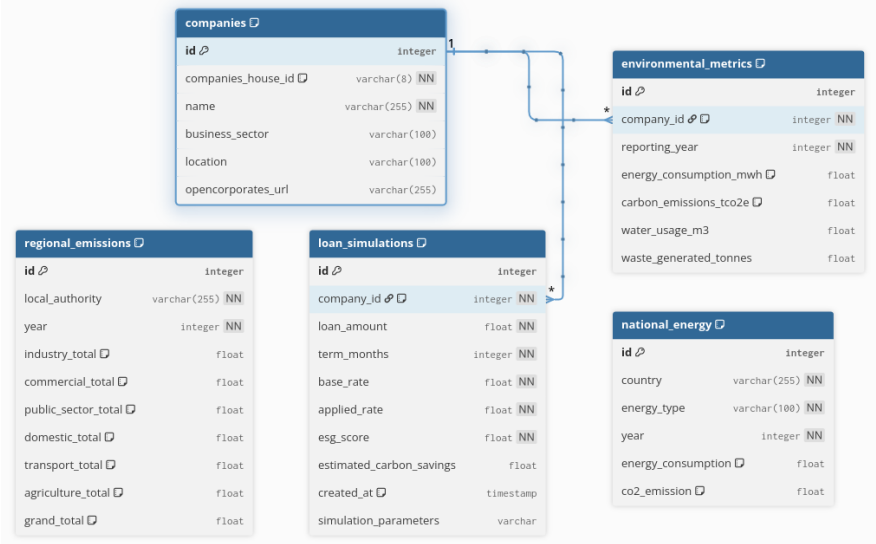

Figure 1: Shows a diagram of the database model relationships within the project .

Furthermore, the schema enforces database-level composite unique constraints (e.g., prohibiting multiple ESG reports for a single company in the same fiscal year) to prevent duplication from client-side errors. All schema changes are version-controlled using Alembic Mike Bayer (2026).

2.2 External Datasets & Integration

To contextualise private corporate inputs, the platform integrates with public, open-source datasets:

1. **OpenCorporates API OpenCorporates Ltd. (2026):** Leveraged for live entity verification during the `POST` ingestion phase. This enforces a hard boundary, preventing the creation of unverified companies by asserting that the provided registration number legally exists within the UK Companies House registry.
2. **UK Government Statistics Department for Energy Security and Net Zero (2024):** Local authority greenhouse gas emissions data acts as the localised baseline within the ESG math engine. An asynchronous database seeding pipeline was engineered with the Pandas data analysis toolkit The pandas development team (2026) to ingest this dataset efficiently into the MRD.

2.3 Algorithmic Engine

The core business logic layer isolates the calculation of the SLL Margin Ratchet from the HTTP network routing tier. When a loan quote is requested, the simulation engine queries the company's submitted metrics alongside the MRD. It executes the ESG discount logic purely in memory, calculating a weighted Environmental Performance Score (EPS). State mutation occurs only upon successful transaction commits, saving the exact parameters as an immutable JSON audit trail on the `LoanSimulation` record to satisfy financial regulatory compliance standards.

3 Testing, DevOps & Error Handling

3.1 Testing Strategy (100% Coverage)

To guarantee the mathematical accuracy of the ESG loan engine and the structural integrity of the PostgreSQL schema, the platform employs a rigorous dual-layer testing suite powered by Pytest Holger Krekel and pytest development team (2026). The suite enforces a strict 80% branch coverage metric, actively monitored via Codecov Sentry (2026) within the CI/CD pipeline.

- **Unit Tests:** Designed for millisecond execution, these tests operate in complete isolation. They use `unittest.mock` to intercept external HTTP calls (e.g., bypassing live OpenCorporates API rate limits) and employ `fakeredis` to validate Cache-Aside logic. Furthermore, they aggressively test the mathematical bounds of the Margin Ratchet engine and the Pydantic schema validation boundaries.
- **Integration Tests:** To validate the interaction between the FastAPI routing layer and the database, the suite uses `pytest-docker` André Caron (2026) to dynamically spawn and teardown ephemeral PostgreSQL and Redis containers. This isolates the test state, allowing for the comprehensive validation of SQLAlchemy transaction rollbacks, database-level composite unique constraints (e.g., rejecting duplicate yearly ESG reports), and the successful execution of cascading `DELETE` operations.

3.2 Error Handling & REST Compliance

The API strictly adheres to industry-standard HTTP status codes and the OpenAPI specification to provide a seamless, predictable developer experience. Expected business logic violations are explicitly caught and handled via FastAPI's `HTTPException` Sebastián Ramírez (2026), returning structured JSON responses. For example, attempting to insert duplicate yearly metrics yields a `409 Conflict`, while querying a deleted entity returns a secure `404 Not Found`.

Furthermore, data malformation is halted at the network perimeter. Pydantic Samuel Colvin (2026) automatically intercepts invalid client payloads (e.g., providing a negative integer for a loan amount) and dynamically generates detailed `422 Unprocessable Entity` errors, preventing malformed data from ever reaching the application's core services.

3.3 Version Control & Containerised Deployment

The project enforces a strict Git feature-branch workflow GitLab (2026), adhering to Semantic Versioning Tom Preston-Werner (2013) and Conventional Commits Conventional Commits (2014) to maintain an auditable history.

To ensure absolute parity between local development and production environments, the ecosystem is fully containerised using Docker Compose Docker Inc. (2026). This decouples the ASGI web server, PostgreSQL database, and Redis cache into isolated microservices. To abstract complex Docker and environment commands, a custom bash orchestrator (`exec.sh`) CSC - IT Center for Science (2026) was engineered to manage the local developer experience.

Finally, deployment is governed by a robust Continuous Integration and Continuous Deployment (CI/CD) pipeline via GitHub Actions GitHub Inc. (2026*a*). The pipeline enforces a "shift-left" security and quality paradigm. On every pull request, automated hooks execute Ruff Astral Software Inc. (2026*a*) for code linting and formatting, Mypy Mypy contributors (2026) for strict static type checking, and the complete Pytest suite. Only upon passing all checks are immutable Docker artifacts packaged and published to the GitHub Container Registry (GHCR) GitHub Inc. (2026*b*).

4 Generative AI Declaration & Analysis

4.1 Declaration of Usage

Generative Artificial Intelligence models assisted throughout the Software Development Life Cycle (SDLC) as pair-programming assistants. Their usage was strictly limited to architectural brainstorming, shell script boilerplate generation, and complex debugging. The core business logic, Domain-Driven Design (DDD) entity modeling, and mathematical algorithms were entirely human-engineered, with AI outputs heavily scrutinised, modified, and reviewed to ensure academic integrity and system security.

4.2 Model Justification & Task Allocation

Model selection was continuously evaluated against current performance and intelligence benchmarks Artificial Analysis (2026) to ensure the appropriate tool was applied to specific developmental tasks.

- **Gemini 1.5 Pro / 3.1 Pro:** Selected for its massive context window and strong reasoning capabilities. It was chosen to digest the FastAPI documentation Sebastián Ramírez (2026) alongside the complete repository file tree, proposing efficient Cache-Aside invalidation patterns and orchestrating the complex multi-container Docker testing scripts.
- **Claude 3.5 Sonnet:** Chosen for its superior Python syntax generation. It assisted in writing the extraction logic to parse the UK Government Excel datasets into the `seed_db.py` loading script, handling complex Pandas The pandas development team (2026) dataframe manipulations.

4.3 Exemplar AI Interaction Logs: The "Zombie PDF" Anomaly

During the End-to-End (E2E) automated testing phase, a critical data-leakage vulnerability was discovered. A company was successfully deleted from the PostgreSQL database, but its previously generated Loan Simulation PDF was still accessible via the API.

Human Prompt (Providing Container Logs):

> *"In my Docker log output, the company was confirmed to be deleted yet it's generated loan simulation pdf is still accessible?:"*

```
$ docker compose logs --tail=3 api
    container | INFO: "DELETE /api/v1/companies/16" 204 No Content
    container | INFO: "GET /api/v1/companies/16" 404 Not Found
    container | INFO: "GET /api/v1/companies/16/simulate-loan/1/pdf" 200 OK

```

Listing 1: Shows the last three api container logs, highlighting a bug.

> *"Why is the PDF endpoint still returning a 200 OK for a deleted company?"*

AI Resolution & Analysis:

> *"You caught a massive, high-severity bug: The Zombie PDF. When you fired the DELETE command, your database cascading logic worked perfectly—the parent company and its metrics were wiped from PostgreSQL (resulting in 404s). However, the PDF binary was Base64 encoded and stored in Redis. When writing the* `delete_company` *route, you forgot to invalidate the loan simulation PDFs. The API found the 'Zombie PDF' in the cache and served it without checking the database. In a banking environment, this is a massive GDPR violation."*

Implemented Solution: The AI proposed a structural fix to the FastAPI `delete_company` routing controller. It suggested executing a SQL `SELECT` query to capture all associated `LoanSimulation.id` values *before* issuing the `db.delete(company)` command. These IDs are then stored in local memory, allowing the server to loop through and explicitly execute `cache.delete()` on every associated Redis key after the database transaction commits. This completely eliminated the data-leakage vulnerability, allowing susbequent tests to pass once more.

5 Critical Reflection & Future Roadmap

5.1 Challenges & Lessons Learned

The most significant architectural challenge encountered during development was reconciling asynchronous database sessions (`asyncpg`) with synchronous, CPU-bound libraries like `ReportLab`. Early iterations of the PDF generation endpoint resulted in severe event-loop blocking, which drastically throttled the FastAPI server's asynchronous throughput.

Resolving this required decoupling the PDF generation into a dedicated, isolated service layer and aggressively caching the binary output via Redis. This provided a critical lesson in system design: in highly concurrent REST architectures, isolating I/O bottlenecks and applying Cache-Aside patterns Bindu B C (2024) is essential for maintaining scalable performance.

5.2 System Limitations

Currently, the system's primary limitation lies in its synchronous dependency on external data providers. During the corporate ingestion phase (`POST /companies`), the OpenCorporates API is invoked synchronously within the HTTP request cycle. Consequently, if the external provider experiences latency or downtime, the endpoint will timeout and fail, degrading the user experience.

Furthermore, to secure the network perimeter, the API currently enforces strict Header-based API Key Authentication (`X-API-Key`). While effective for B2B machine-to-machine communication, without a dedicated API Gateway to handle dynamic rate-limiting or native OAuth2 integration, the system remains constrained as a single-tenant environment.

5.3 Strategic Roadmap & Future Enhancements

To evolve this Banking-as-a-Service (BaaS) simulator into a production-grade enterprise application, the following developmental phases are proposed:

1. **Event-Driven Architecture (EDA):** Implementing a message broker (e.g., RabbitMQ or Celery) to offload heavy operations. By pushing PDF rendering and external API validation to background worker queues, the API could immediately return a `202 Accepted` status, greatly improving client perceived performance.

2. **Deployment, Publishing & Hosting:** Transitioning the project from local Docker Compose environments to managed cloud infrastructure. Makinguse of the immutable artifacts already published to the GitHub Container Registry (GHCR) GitHub Inc. (2026*b*), the API could be deployed to Platform-as-a-Service (PaaS) providers like Railway **?**. Furthermore, exposing the API via the Model Context Protocol (MCP) Anthropic (2026) would allow Large Language Models to natively query the loan engine as a tool.

3. **Client-Facing Frontend & RBAC:** Developing a modern web dashboard (e.g., React or Next.js) for bank administrators. This would necessitate upgrading the static API Key perimeter to granular Role-Based Access Control (RBAC) via JSON Web Tokens (JWTs), enforcing multi-tenancy so users can only view their verified organisational entities.

4. **Dataset Expansion & Algorithmic Maturity:** Broadening the ESG mathematical engine by integrating wider, real-time datasets. Moving beyond basic regional `CO2e` baselines to include global supply chain analytics (Scope 3 emissions) and real-time national grid carbon intensity APIs.

Bibliography

André Caron (2026), 'pytest-docker: Docker-based integration tests', Available from: https://pypi.org/project/pytest-docker/. Accessed: 2026-02-26.

Anthropic (2026), 'Model context protocol (mcp) documentation', Available from: https://modelcontextprotocol.io/docs/getting-started/intro. Accessed: 2026-02-26.

Artificial Analysis (2026), 'Comparison of models: Intelligence, performance & price analysis', Available from: https://artificialanalysis.ai/models. Accessed: 2026-02-26.

Astral Software Inc. (2026*a*), 'Ruff: An extremely fast python linter and code formatter, written in rust', Available from: https://docs.astral.sh/ruff/. Accessed: 2026-03-12.

Astral Software Inc. (2026*b*), 'uv: An extremely fast python package and project manager, written in rust', Available from: https://github.com/astral-sh/uv. Accessed: 2026-03-09.

Bindu B C (2024), 'System design: Cache-aside pattern implementation', Available from: https://medium.com/@bindubc/system-design-cache-aside-pattern-implementation-bf433a5c0dc9. Accessed: 2026-02-26.

Conventional Commits (2014), 'Conventional commits 1.0.0', Available from: https://www.conventionalcommits.org/en/v1.0.0/. Accessed: 2026-02-26.

CSC - IT Center for Science (2026), 'Linux bash scripts', Available from: https://docs.csc.fi/support/tutorials/env-guide/linux-bash-scripts/. Accessed: 2026-02-26.

Department for Energy Security and Net Zero (2024), 'Uk local authority and regional greenhouse gas emissions statistics', Available from: https://www.gov.uk/government/statistics/uk-local-authority-and-regional-greenhouse-gas-emissions-statistics-2005-to-2023. Accessed: 2026-02-26.

Docker Inc. (2026), 'Docker compose overview', Available from: https://docs.docker.com/compose/. Accessed: 2026-03-12.

Encode OSS (2026), 'Uvicorn: The lightning-fast asgi server', Available from: https://www.uvicorn.org/. Accessed: 2026-03-09.

Fırat Özcan (2024), 'Transitioning to microservices with django: From monolith to microservices', Available from: https://medium.com/@firatozc555/transitioning-to-microservices-with-django-from-monolith-to-microservices-91356528fc02. Accessed: 2026-02-26.

GitHub Inc. (2026*a*), 'Events that trigger workflows - github actions', Available from: https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows. Accessed: 2026-02-26.

GitHub Inc. (2026*b*), 'Working with the container registry', Available from: https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry. Accessed: 2026-02-26.

GitLab (2026), 'Version control best practices', Available from: https://about.gitlab.com/topics/version-control/version-control-best-practices/. Accessed: 2026-02-26.

Hitorunajp (2025), 'Poetry vs. uv: Which python package manager should you use in 2025?', Available from: https://medium.com/@hitorunajp/poetry-vs-uv-which-python-package-manager-should-you-use-in-2025-4212cb5e0a14. Accessed: 2026-03-09.

Holger Krekel and pytest development team (2026), 'pytest: helps you write better programs', Available from: https://docs.pytest.org/. Accessed: 2026-02-26.

Level Up Coding (2024), 'Caching data with redis and sqlalchemy in python: A step-by-step guide', Available from: https://levelup.gitconnected.com/caching-data-with-redis-and-sqlalchemy-in-python-a-step-by-step-guide-97f898f55ef. Accessed: 2026-03-12.

Loopwerk (2024), 'Python package management: Poetry vs uv', Available from: https://www.loopwerk.io/articles/2024/python-poetry-vs-uv/. Accessed: 2026-03-09.

MagicStack Inc. (2026), 'asyncpg: A fast postgresql database client library for python/asyncio', Available from: https://magicstack.github.io/asyncpg/current/. Accessed: 2026-03-09.

Mike Bayer (2026), 'Alembic database migration tool', Available from: https://alembic.sqlalchemy.org/en/latest/. Accessed: 2026-03-09.

Mike Bayer and SQLAlchemy authors (2026*a*), 'Asynchronous i/o (asyncio) — sqlalchemy 2.1 documentation', Available from: https://docs.sqlalchemy.org/en/21/orm/extensions/asyncio.html. Accessed: 2026-02-26.

Mike Bayer and SQLAlchemy authors (2026*b*), 'Sqlalchemy 2.0 documentation', Available from: https://docs.sqlalchemy.org/en/20/. Accessed: 2026-03-09.

Mypy contributors (2026), 'mypy documentation', Available from: https://mypy.readthedocs.io/en/stable/. Accessed: 2026-02-26.

OpenAPI Initiative (2023), 'Openapi specification v3.1.0', Available from: https://spec.openapis.org/oas/v3.1.0. Accessed: 2026-03-12.

OpenCorporates Ltd. (2026), 'Opencorporates api documentation', Available from: https://opencorporates.com/info/api. Accessed: 2026-02-26.

Practical Law Finance (2026), 'Sustainability-linked loan provisions', Available from: https://uk.practicallaw.thomsonreuters.com/w-035-2624. Accessed: 2026-02-26.

Python Software Foundation (2024), 'Python 3.12 documentation', Available from: https://docs.python.org/3.12/. Accessed: 2026-03-12.

Redis Ltd. (2026), 'Redis development documentation', Available from: https://redis.io/docs/latest/develop/. Accessed: 2026-02-26.

ReportLab Europe Ltd. (2026), 'Reportlab pdf library', Available from: https://docs.reportlab.com/. Accessed: 2026-03-12.

Samuel Colvin (2026), 'Pydantic: Data validation using python type hints', Available from: https://docs.pydantic.dev/latest/. Accessed: 2026-03-09.

Sebastián Ramírez (2026), 'Fastapi framework documentation', Available from: https://fastapi.tiangolo.com/. Accessed: 2026-03-09.

Sentry (2026), 'Codecov documentation', Available from: https://docs.codecov.com/docs/. Accessed: 2026-02-26.

Stoplight (2026), 'Api types: What are they and why do they matter?', Available from: https://stoplight.io/api-types. Accessed: 2026-02-26.

The pandas development team (2026), 'pandas: powerful python data analysis toolkit', Available from: https://pandas.pydata.org/docs/. Accessed: 2026-03-12.

The PostgreSQL Global Development Group (2026), 'Postgresql documentation', Available from: https://www.postgresql.org/docs/. Accessed: 2026-02-26.

Tom Preston-Werner (2013), 'Semantic versioning 2.0.0', Available from: https://semver.org/. Accessed: 2026-02-26.

1 Appendix: Generative AI Conversation Logs

The following transcripts document the exact interactions with Generative AI (Gemini 3.1 Pro) used to resolve complex architectural and environmental challenges during the project's development.

A.1 Architectural Implementation: API Security Perimeter

Context: The initial API draft lacked a security perimeter. AI was utilized to brainstorm an enterprise-appropriate, lightweight authentication mechanism suitable for a B2B microservice.

Human Prompt:

> *"Seeing as this is a simple backe-end API, would a front-end interface with an authentication layer be appropriate?"*

AI Resolution & Analysis:

> *"For a B2B Banking-as-a-Service platform, implementing a full OAuth2/JWT user-management system right now would be overkill and bloat your clean architecture. However, the industry standard for a 'neat, efficient, and simple' B2B authentication method is a Static API Key passed via an `X-API-Key` HTTP header (exactly like how Stripe or OpenCorporates secures their endpoints).*
>
> *Here is an example implementation using FastAPI's native security utilities, which will automatically integrate with the Swagger UI:"*

```
from fastapi import HTTPException, Security
from fastapi.security import APIKeyHeader

api_key_header = APIKeyHeader(name="X-API-Key", auto_error=False)

async def verify_api_key(api_key = Security(api_key_header)):
    if not api_key or api_key != API_KEY:
        raise HTTPException(
            status_code=401,
            detail="Invalid or Missing API Key"
        )
    return api_key
```

A.2 Complex Debugging: The "Zombie PDF" Cache Desynchronization

Context: Automated integration tests revealed that simulated SLL PDFs were still accessible via the REST endpoint even after the parent company had been hard-deleted from the PostgreSQL database.

Human Prompt (Providing Container Logs):

> *"In my Docker log output, the company was confirmed to be deleted yet its generated loan simulation pdf is still accessible:*
> *`green-fintech-api | INFO: "DELETE /api/v1/companies/16" 204 No Content`*
> *`green-fintech-api | INFO: "GET /api/v1/companies/16" 404 Not Found`*
> *`green-fintech-api | INFO: "GET /api/v1/companies/16/simulate-loan/1/pdf" 200 OK`*
> *Why is the PDF endpoint still returning a 200 OK for a deleted company?"*

AI Resolution & Analysis:

> *"You caught a massive, high-severity bug: The Zombie PDF. When you fired the DELETE command, your database cascading logic worked perfectly—the parent company and its metrics were wiped from PostgreSQL (resulting in 404s). However, the PDF binary was Base64 encoded*

and stored in Redis. When writing the `delete_company` route, you forgot to invalidate the loan simulation PDFs. The API found the 'Zombie PDF' in the cache and served it without checking the database. In a banking environment, this is a massive GDPR violation.

To fix this, execute a SQL `SELECT` query to capture all associated `LoanSimulation.id` values ***before*** *issuing the `db.delete(company)` command. Store these IDs in local memory, allowing the server to loop through and explicitly execute `cache.delete()` on every associated Redis key after the database transaction commits."*

The following transcripts document the exact interactions with Generative AI (Claude 3.5 Sonnet) used to resolve redundancy issues and debug workflows during the project's development.

A.3 CI/CD Deployment Debugging: GHCR Naming Convention Failure

Context: The GitHub Actions CI/CD pipeline repeatedly failed during the `docker push` step when attempting to publish the built artifact to the GitHub Container Registry (GHCR).

Human Prompt:

"My GitHub Actions workflow is failing on the 'Push Docker Image' step. The error says `invalid reference format: repository name must be lowercase`. My GitHub repository is named `Green-FinTech-BaaS`. How do I fix this in the YAML file without renaming my entire repository?"

AI Resolution & Analysis:

"Docker strictly enforces lowercase repository names for all image tags, but the default `${{ github.repository }}` variable in GitHub Actions preserves the exact casing of your repository name (`Green-FinTech-API`). Attempting to push this mixed-case string to `ghcr.io` throws the invalid reference error.

You can resolve this dynamically by adding a bash text-transformation step using `tr` to convert the repository name to lowercase before the Docker build step, storing it securely in the `GITHUB_ENV` file for subsequent steps to use."

```
- name: Set lowercase repository name
  run: |
    echo "REPO_LC=${GITHUB_REPOSITORY,,}" >> ${GITHUB_ENV}

- name: Build and push Docker image
  uses: docker/build-push-action@v5
  with:
    push: true
    tags: ghcr.io/${{ env.REPO_LC }}:latest
```

A.4 Architecture Refactoring: Bash Orchestrator Decomposition

Context: As the project scaled, the local developer orchestration script (`exec.sh`) grew to over 500 lines, handling Docker logic, PostgreSQL seeding, Redis diagnostics, and API testing in a single file, making it difficult to maintain.

Human Prompt:

"My `exec.sh` orchestrator is over 500 lines long and handles docker, postgres, redis, and api testing all in one file. How can I cleanly decompose and modularise this while sharing variables like `HOST_URL`, the UI formatting colours and the loading of .env file variables which I need at every step?"

AI Resolution & Analysis:

> *"A monolithic 500-line bash script is an anti-pattern. You need to apply a modular architecture similar to your Python code. Extract all shared configurations (like URLs, colors, and the `load_env` function) into a standalone `common.sh` file.*
>
> *Then, create domain-specific helpers (`api-helper.sh`, `pg-helper.sh`, `rd-helper.sh`) inside your existing `scripts/` directory. Then create a `common.sh` file to be dynamically injected into these helpers by using robust relative path sourcing, ensuring the scripts work regardless of where the developer executes them from."*

```
#!/bin/bash
# scripts/common.sh

set -e

# --- Dynamic Path Resolution ---
# Safely resolve the project root regardless of where the script is executed from
PROJECT_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"

# --- Configuration & Colors ---
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
ORANGE='\033[0;33m'
BLUE='\033[0;34m'
PURPLE='\033[0;35m'
BOLD='\033[1m'
NC='\033[0m'

COMPOSE_CMD="docker compose"

HOST_URL="http://localhost:8080/api/v1/"
ROOT_URL="http://localhost:8080"

# --- UI Helpers ---
log_warn()      { echo -e " ${YELLOW}${BOLD}X${YELLOW} $1${NC}"; }
log_serious()   { echo -e " ${ORANGE}${BOLD}X${ORANGE} $1${NC}"; }
log_error()     { echo -e " ${RED}${BOLD}X${RED} $1${NC}"; }
log_success()   { echo -e " ${GREEN}${BOLD}T${NC} ${GREEN}$1${NC}"; }
log_data()      { echo -e " ${NC}${BOLD}  >${NC} $1${NC}"; }
log_info()      { echo -e " ${BLUE}${BOLD}i${BLUE} $1${NC}"; }
header()        { echo -e "\n${PURPLE}${BOLD}=========== $1 ===========${NC}"; }

assert_cmd() {
    local success_msg="$1"
    local error_msg="$2"
    shift 2
    if "$@"; then
        [ -n "$success_msg" ] && log_success "$success_msg"
    else
        [ -n "$error_msg" ] && log_error "$error_msg"
        exit 1
    fi
}
```

```
ask_yes_no() {
    read -p "$(echo -e "${YELLOW} ? $1 (y/N): ${NC}")" response
    case "$response" in
        [yY][eE][sS]|[yY]) return 0 ;;
        *) return 1 ;;
    esac
}

# --- Environment Setup ---
load_env() {
    local env_file="${PROJECT_ROOT}/.env"
    if [ -f "$env_file" ]; then
        while IFS= read -r line || [ -n "$line" ]; do
            if [[ ! "$line" =~ ^\s*# ]] && [[ -n "$line" ]]; then
                clean_line=$(echo "$line" | sed 's/\s*#.*$//' | tr -d '\r')
                export "$clean_line"
            fi
        done < "$env_file"
    fi
    export DATABASE_URL="postgresql+asyncpg://${POSTGRES_USER}:${POSTGRES_PASSWORD}@localhost:${
}

# --- Generic Docker/Compose Wrappers ---
_compose_up()       { $COMPOSE_CMD up -d "$@" }
_compose_build_up() { $COMPOSE_CMD up --build -d "$@" }
_compose_stop()     { $COMPOSE_CMD stop "$@" }
_compose_down()     { $COMPOSE_CMD down "$@" }
_compose_wipe()     { $COMPOSE_CMD down --remove-orphans -v "$@" }
_compose_logs() {
    local lines=${2:-20}
    $COMPOSE_CMD logs --tail="$lines" "$1"
}

# other shared commands...
```

Then, you can import it in your helpers...

```
#!/bin/bash
# scripts/api-helper.sh

set -e

# --- Configuration ---
# Import the common library relative to this script's location
source "$(dirname "${BASH_SOURCE[0]}")/common.sh"

show_env() {
    echo
    log_info "Current API Configuration:"
    log_data "Container: ${API_CONTAINER}"
    log_data "Root URL (${API_PORT}): ${ROOT_URL}"  # <-- Uses the imported variables
    log_data "Host URL (V1): ${HOST_URL}"
    log_data "Output Dir: ${OUT_DIR}"
}

# --- API Curl Wrappers ---
```

```
_api_get() {
    local endpoint="${1#/}"
    curl -s -X 'GET' "${HOST_URL}${endpoint}" \
        -H "X-API-Key: ${API_KEY}" \
        -H 'accept: application/json'
}

# and so on...
```